ByteMart Digital

THE INTERVIEW PREP SERIES

REST API Interview Q&A

REST principles, HTTP methods, status codes, URI design, security, caching & coding

119 interview questions with concise, accurate model answers, including 6 hands-on coding challenges with working solutions — organized by topic for focused preparation.

119	6	15	Model
Questions	Coding	Sections	Answers

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

REST API Interview Questions

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided in good faith for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from its use. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. All other product names and brands are the property of their respective owners and are used for identification purposes only. This is an independent work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

REST Fundamentals 4

HTTP Methods & CRUD 7

Status Codes 10

URI & Resource Design 12

Design Best Practices 14

Security 16

Performance & Caching 19

Documentation & Standards 21

REST vs Alternatives 23

Advanced Topics 25

HTTP Semantics Deep-Dive 28

Design & Best Practices Extras 30

Security, Testing & Operations 32

Practical Scenarios 34

Coding Challenges 38

This bank contains 119 REST API interview questions across 15 sections, including 6 coding challenges with full solutions. Use the bookmarks panel to jump to any section or question.

SECTION 01

REST Fundamentals

Q1 What is REST?

ANSWER

REST (Representational State Transfer) is an architectural style for networked applications, defined by a set of constraints. A RESTful API exposes **resources** (identified by URLs) that clients act on using standard HTTP methods, exchanging **representations** (usually JSON). It is not a protocol or a standard, but a set of design principles that leverage HTTP as it was intended.

> **KEY IDEA**
> REST models a system as resources addressed by URLs and manipulated with standard HTTP methods and status codes — stateless, uniform, and cacheable.

Q2 What are the six architectural constraints of REST?

ANSWER

- **Client-server** — separation of concerns between UI and data storage.
- **Stateless** — each request carries all information needed; the server keeps no client session.
- **Cacheable** — responses declare whether they can be cached.
- **Uniform interface** — a consistent way to address and manipulate resources.
- **Layered system** — intermediaries (proxies, gateways) can sit between client and server transparently.
- **Code on demand** (optional) — servers may send executable code to clients.

Satisfying these (the uniform interface especially) is what makes an API genuinely RESTful rather than merely HTTP-based.

Q3 What is statelessness, and why does it matter?

ANSWER

Statelessness means the server stores no client context between requests — every request must contain all the information needed to process it (credentials, parameters). This makes REST APIs easy to scale horizontally (any instance can handle any request), more resilient (no session to lose), and simpler to reason about.

> **WATCH OUT**
> Storing session state on the server breaks statelessness and complicates scaling; carry state in the request (e.g. a token) or in the client instead.

© 2026 ByteMart Digital

Q4 What is a resource, and how does it differ from its representation?

ANSWER

A **resource** is any addressable concept in your domain — an order, a user, a collection of products — identified by a URI. A **representation** is a concrete serialisation of that resource's current state at a point in time, such as a JSON or XML document. The same resource can have multiple representations (JSON, XML) negotiated via headers.

Q5 What is the uniform interface constraint?

ANSWER

The uniform interface is REST's defining constraint. It means resources are identified in requests (by URI), manipulated through their representations, messages are self-descriptive (headers convey how to process them), and — ideally — hypermedia drives state transitions (HATEOAS). A consistent, predictable interface is what lets clients and servers evolve independently.

Q6 What is HATEOAS?

ANSWER

HATEOAS (Hypermedia As The Engine Of Application State) means responses include **links** to related actions and resources, so a client discovers what it can do next from the response itself rather than hard-coding URLs. It is the highest level of REST maturity, decoupling clients from URL structures, though in practice many APIs adopt it only partially.

```json
{
  "id": 42, "status": "PENDING",
  "_links": {
    "self":   { "href": "/orders/42" },
    "cancel": { "href": "/orders/42/cancel" }
  }
}
```

Q7 What is the Richardson Maturity Model?

ANSWER

It grades an API's 'RESTfulness' in four levels. **Level 0** — a single URI, HTTP as a tunnel (RPC-style). **Level 1** — multiple resources with distinct URIs. **Level 2** — proper use of HTTP verbs and status codes (where most good APIs sit). **Level 3** — hypermedia controls (HATEOAS). Each level adds discipline toward a fully uniform interface.

© 2026 ByteMart Digital

Q8 What makes an API RESTful versus just an HTTP API?

ANSWER

Many APIs use HTTP but are not RESTful — for example, tunnelling everything through POST to a single endpoint with verbs in the body. A RESTful API honours the constraints: resource-based URIs, correct HTTP methods and status codes, statelessness, and a uniform interface. The distinction is about respecting HTTP's semantics rather than merely transporting data over it.

SECTION 02

HTTP Methods & CRUD

Q9 How do HTTP methods map to CRUD operations?

ANSWER

Method	CRUD	Purpose
GET	Read	retrieve a resource (no side effects)
POST	Create	create a new resource
PUT	Update	replace a resource entirely
PATCH	Update	partially modify a resource
DELETE	Delete	remove a resource

Using the right method for each operation is central to REST — it makes the API predictable and lets HTTP infrastructure (caches, proxies) behave correctly.

Q10 What is the difference between safe and idempotent methods?

ANSWER

A **safe** method does not modify server state — `GET`, `HEAD`, `OPTIONS`. An **idempotent** method produces the same result no matter how many times it is repeated — `GET`, `PUT`, `DELETE`. All safe methods are idempotent, but not all idempotent methods are safe.

Method	Safe	Idempotent
GET	yes	yes
PUT	no	yes
DELETE	no	yes
POST	no	no
PATCH	no	not necessarily

KEY IDEA

Idempotency matters for retries — a client can safely retry an idempotent request after a timeout without risking duplicate side effects.

© 2026 ByteMart Digital

Q11 What is the difference between PUT and PATCH?

ANSWER

PUT **replaces** the entire resource with the representation sent — omitted fields are typically cleared — and is idempotent. PATCH applies a **partial** update, changing only the supplied fields. Use PUT for full replacement and PATCH for targeted edits.

```text
PUT /users/42        -> send the whole user; missing fields reset
PATCH /users/42      -> { "email": "new@x.com" } changes only email
```

Q12 What is the difference between PUT and POST for creation?

ANSWER

POST creates a resource where the **server** assigns the identifier, so repeating it creates duplicates (not idempotent). PUT creates or replaces a resource at a **client-specified** URI, so repeating it has the same effect (idempotent). Use POST to a collection for server-generated ids; use PUT when the client controls the id.

Q13 What are the HEAD and OPTIONS methods for?

ANSWER

HEAD is like GET but returns only headers, no body — useful for checking existence, size, or freshness cheaply. OPTIONS asks which methods and capabilities an endpoint supports, and is used by browsers for CORS preflight requests.

Q14 Is POST ever idempotent, and how can you make it so?

ANSWER

POST is not idempotent by default — two identical creates make two resources. You can make it effectively idempotent with an **idempotency key**: the client sends a unique key, the server records it, and a retry with the same key returns the original result instead of creating a duplicate. This is standard for payment and order APIs.

© 2026 ByteMart Digital

Q15 What method should a 'search' or complex query use?

ANSWER

Simple filters belong in a `GET` with query parameters, keeping the request safe and cacheable. When a query is too large or complex for a URL (long filter payloads), a pragmatic pattern is a `POST` to a `/search` sub-resource carrying the criteria in the body — accepting the loss of caching in exchange for expressiveness.

Q16 Why should GET never change server state?

ANSWER

`GET` is defined as safe, and the entire HTTP ecosystem relies on that: browsers prefetch links, caches store responses, and crawlers follow URLs — all assuming `GET` has no side effects. A `GET` that mutates state can be triggered accidentally and repeatedly by this infrastructure, causing real damage.

> **WATCH OUT**
> Never put a state-changing action behind a `GET` (like `/orders/42/delete`) — a prefetch or crawler could execute it.

SECTION 03

Status Codes

Q17 What do the five status code classes mean?

ANSWER

Class	Meaning
1xx	informational (rarely used directly)
2xx	success
3xx	redirection
4xx	client error (the request was wrong)
5xx	server error (the server failed)

Returning a status code from the correct class is essential — it tells clients and infrastructure how to react (retry, redirect, fix the request).

Q18 What is the difference between 200, 201, and 204?

ANSWER

200 OK is general success with a response body. **201 Created** signals a new resource was created, typically with a `Location` header pointing to it. **204 No Content** is success with no body — ideal for a `DELETE` or an update that returns nothing.

Q19 What is the difference between 400 and 422?

ANSWER

400 Bad Request means the request was malformed — invalid JSON, wrong types, unparseable. **422 Unprocessable Entity** means the request was syntactically valid but failed **semantic** validation — e.g. a required business rule was violated. Many APIs use 400 for both, but 422 more precisely describes validation failures.

Q20 What is the difference between 401 and 403?

ANSWER

401 Unauthorized means authentication is missing or invalid — the server does not know who you are ('log in'). **403 Forbidden** means you are authenticated but not allowed to perform this action — the server knows who you are and refuses ('you cannot do this').

© 2026 ByteMart Digital

KEY IDEA
401 is about identity (who are you?), 403 is about permission (are you allowed?).

Q21 What is the difference between 404 and 410?

ANSWER

404 Not Found means the resource does not exist (or is hidden), with no implication about its history. **410 Gone** explicitly states the resource existed but has been permanently removed, signalling clients and search engines to stop requesting it.

Q22 When would you return 409 Conflict?

ANSWER

409 Conflict indicates the request conflicts with the current state of the resource — a duplicate creation, a version mismatch in optimistic locking, or a business-rule conflict. It tells the client the operation cannot proceed because of a state clash, often prompting a reload-and-retry.

Q23 What is 429, and what header should accompany it?

ANSWER

429 Too Many Requests is returned when a client exceeds a rate limit. It should include a `Retry-After` header telling the client how long to wait before retrying, and often rate-limit headers describing the quota and remaining allowance.

Q24 What is the difference between 500 and 503?

ANSWER

500 Internal Server Error is a generic, unexpected server failure (an unhandled exception). **503 Service Unavailable** means the server is temporarily unable to handle the request — overloaded or down for maintenance — and is typically transient, so it may include `Retry-After` and invites a retry.

WATCH OUT
Never leak stack traces or internal details in a 500 response — return a generic, structured error and log the specifics server-side.

© 2026 ByteMart Digital

SECTION 04

URI & Resource Design

Q25 What are the conventions for naming REST URIs?

ANSWER

- Use **nouns**, not verbs — `/orders`, not `/getOrders`. The HTTP method is the verb.
- Use **plural** collection names — `/users/42`, not `/user/42`.
- Use lowercase, hyphen-separated paths — `/order-items`.
- Express hierarchy with nesting — `/orders/42/items`.
- Keep query concerns (filter, sort, paginate) in query parameters.

WATCH OUT

Putting verbs in URIs (`/createOrder`, `/deleteUser`) is the most common REST design mistake — it duplicates what the HTTP method already conveys.

Q26 How should you design nested / sub-resources?

ANSWER

Nest a resource under its parent when it only exists in that context — `/orders/42/items` for the items of order 42. Avoid nesting more than one or two levels deep, since long chains become unwieldy; beyond that, expose the sub-resource at a top level and filter by parent id (`/items?orderId=42`).

Q27 How do you handle filtering, sorting, and pagination?

ANSWER

Use query parameters on the collection endpoint: filtering (`?status=paid`), sorting (`?sort=createdAt,desc`), and pagination (`?page=0&size;=20` or cursor-based). Keeping these in query parameters preserves the `GET`'s safety and cacheability.

```text
GET /orders?status=paid&sort=createdAt,desc&page=0&size=20
```

Q28 What is the difference between offset and cursor pagination?

ANSWER

Offset pagination (`page/size` or `limit/offset`) is simple and allows jumping to any page, but it is slow on large datasets and can skip or repeat rows when data changes mid-paging. **Cursor** (keyset) pagination uses a pointer to the last item on an indexed column — efficient and stable for large, changing datasets, but only allows sequential navigation.

Q29 How do you design an action that isn't a simple CRUD operation?

ANSWER

Some operations (approve, cancel, publish) do not map cleanly to CRUD. Model them either as a state change via `PATCH` to the resource (`{ "status": "CANCELLED" }`), or as a sub-resource action with `POST` (`POST /orders/42/cancel`). The action-as-sub-resource style is a pragmatic, widely-used convention.

Q30 Should collection names be singular or plural?

ANSWER

Use **plural** nouns consistently — `/products` for the collection and `/products/42` for one item. Plural reads naturally for both the collection and its members and is the dominant industry convention. The key is consistency across the whole API.

Q31 How do you represent relationships between resources?

ANSWER

Represent a relationship either by nesting (`/orders/42/items`), by including the related id or a link in the representation, or by a query filter. For many-to-many or independently-accessed relations, top-level resources with links are cleaner than deep nesting; HATEOAS links make the relationships navigable.

Q32 What is the difference between a collection resource and a singleton resource?

ANSWER

A **collection** resource represents a set of items (`/orders`) supporting listing and creation; an individual item is addressed within it (`/orders/42`). A **singleton** resource represents a single, unique instance with no collection (`/users/me/profile`). Choosing the right shape keeps the API intuitive.

© 2026 ByteMart Digital

SECTION 05

Design Best Practices

Q33 How should you version a REST API?

ANSWER

Version only for breaking changes, evolving additively otherwise. The main strategies are URI versioning (`/v1/orders`), header versioning (a custom or `Accept` header), and content-type versioning. URI versioning is the most visible, cache-friendly, and easiest to route, which makes it the common default.

Strategy	Example
URI	/v2/orders
Header	Accept: application/vnd.api.v2+json
Query param	/orders?version=2

Q34 What makes a good error response?

ANSWER

A good error response uses the correct status code and a consistent, structured body containing a machine-readable code, a human-readable message, and (for validation) field-level details — never a stack trace. The RFC 7807 `ProblemDetail` format standardises this.

```json
{
  "type": "https://api.bytemart.com/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "errors": [ { "field": "email", "message": "must be a valid email" } ]
}
```

Q35 Should responses use an envelope?

ANSWER

An **envelope** wraps the payload in a standard structure (`{ "data": ..., "meta": ... }`), which is handy for attaching pagination metadata and a consistent error shape. The alternative returns the resource directly and puts metadata in headers/links, which is cleaner and more HTTP-native. Either is fine — consistency across the API matters most.

Q36 What is content negotiation in REST?

ANSWER

Content negotiation lets a client request a specific representation via the `Accept` header (e.g. `application/json` vs `application/xml`), and the server responds with a matching `Content-Type`. It allows one resource to be served in multiple formats and versions without changing its URI.

Q37 Why is consistency important in API design?

ANSWER

A consistent API — uniform naming, error formats, pagination, date formats, and conventions across every endpoint — is dramatically easier to learn and integrate against, because a client can predict how any endpoint behaves from experience with others. Inconsistency forces developers to re-learn each endpoint and is a frequent source of bugs.

Q38 How should you handle date and time in APIs?

ANSWER

Use a single, unambiguous standard — ISO 8601 in UTC (`2026-07-07T10:30:00Z`) — for all timestamps. This avoids timezone confusion and locale-specific formats, and lets clients convert to local time as needed. Consistency here prevents a whole class of subtle bugs.

Q39 What is an idempotency key, and when should an API support it?

ANSWER

An idempotency key is a unique value the client sends (usually as a header) so the server can detect and de-duplicate retried requests, returning the original result rather than repeating a side effect. APIs for payments, orders, and any non-idempotent `POST` should support it so clients can retry safely after network failures.

Q40 What is the principle of least surprise in API design?

ANSWER

An API should behave the way a competent developer would expect — standard status codes, conventional URIs, predictable pagination, and consistent naming — so integrators rarely need to consult documentation for basic behaviour. Following established conventions rather than inventing novel ones is the surest way to a usable API.

SECTION 06

Security

Q41 Why must REST APIs use HTTPS?

ANSWER

HTTPS encrypts the entire request and response in transit, protecting credentials, tokens, and data from interception and tampering on the network. Without it, bearer tokens and API keys travel in plain text and can be captured. HTTPS is the non-negotiable baseline for any API handling authentication or sensitive data.

> **KEY IDEA**
> HTTPS everywhere is the foundation — every other API security measure is undermined if traffic can be read or altered in transit.

Q42 What are the common authentication schemes for REST APIs?

ANSWER

- **Basic auth** — username:password base64-encoded in the header; simple but must be over HTTPS and is rarely used alone in production.
- **API keys** — a shared secret identifying the caller; good for server-to-server, but coarse-grained.
- **Bearer tokens (JWT)** — a signed token in the `Authorization` header; stateless and the common choice.
- **OAuth2 / OIDC** — delegated authorization for third-party access, the standard for user-facing platforms.

Q43 How does token-based (JWT) authentication work in a REST API?

ANSWER

The client authenticates once and receives a signed JWT containing claims (identity, roles, expiry). It sends the token in the `Authorization: Bearer` header on each request; the server validates the signature and expiry and authorises from the claims — no server-side session needed, which suits stateless REST and horizontal scaling.

> **WATCH OUT**
> JWTs cannot be easily revoked before expiry, so keep lifetimes short and pair with refresh tokens; never store sensitive data in the payload, which is only base64-encoded, not encrypted.

© 2026 ByteMart Digital

Q44 What is CORS, and what problem does it solve?

ANSWER

Cross-Origin Resource Sharing is a browser security mechanism that controls whether a web page from one origin may call an API on another. The server declares allowed origins, methods, and headers via `Access-Control-Allow-*` response headers; for non-simple requests the browser first sends an `OPTIONS` preflight. CORS relaxes the same-origin policy in a controlled way.

> **WATCH OUT**
> Do not set `Access-Control-Allow-Origin: *` for APIs that use credentials — specify exact trusted origins instead.

Q45 What is the difference between authentication and authorization?

ANSWER

Authentication verifies who the caller is (validating credentials or a token). **Authorization** determines what they are allowed to do (checking roles or permissions). Authentication comes first; authorization then gates each action. In HTTP terms, failing authentication yields 401, failing authorization yields 403.

Q46 What is broken object-level authorization, and how do you prevent it?

ANSWER

It is the most common API vulnerability (OWASP API #1): an authenticated user accesses another user's data by changing an id in the URL (`/orders/43` instead of their own `/orders/42`), because the server checks that they are logged in but not that the resource belongs to them. Prevent it by verifying ownership on every object access, not just authentication.

```java
Order o = repo.findById(id).orElseThrow(NotFoundException::new);
if (!o.ownerId().equals(currentUserId()))
    throw new ForbiddenException();   // ownership check, not just auth
```

Q47 How do you protect an API against injection attacks?

ANSWER

Validate and sanitise all input, use parameterised queries or an ORM (never string-concatenated SQL), and encode output appropriately. Injection happens when untrusted input is interpreted as code or a query; treating all client input as untrusted and never mixing it into commands is the defence.

Q48 Why is input validation important at the API boundary?

ANSWER

The API boundary is where untrusted external data enters the system, so validating types, ranges, formats, and required fields there prevents malformed or malicious data from reaching business logic and storage. It also returns clear, early errors to clients. Validation should be explicit and fail closed — reject anything not known to be valid.

SECTION 07

Performance & Caching

Q49 How does HTTP caching work with Cache-Control?

ANSWER

The `Cache-Control` response header tells clients and intermediaries how a response may be cached — `max-age` (how long it stays fresh), `public/private` (who may cache it), and `no-store/no-cache` (do not cache / revalidate first). Proper caching dramatically cuts load and latency for read-heavy APIs.

```text
Cache-Control: public, max-age=300     # cacheable for 5 minutes
Cache-Control: no-store                # never cache (sensitive data)
```

Q50 What is an ETag, and how does conditional GET work?

ANSWER

An `ETag` is a version identifier (often a hash) the server returns for a resource. On the next request the client sends it back in `If-None-Match`; if the resource is unchanged, the server replies **304 Not Modified** with no body, saving bandwidth. It enables efficient revalidation without re-transferring unchanged data.

```text
GET /orders/42            -> 200 OK, ETag: "a1b2c3"
GET /orders/42
If-None-Match: "a1b2c3"   -> 304 Not Modified (no body)
```

Q51 What is the difference between ETag and Last-Modified?

ANSWER

Both support conditional requests, but `Last-Modified` uses a timestamp (paired with `If-Modified-Since`) while `ETag` uses an opaque version token (paired with `If-None-Match`). ETags are more precise — they detect any change, even within the same second, and handle content that changes without a clear modification time.

© 2026 ByteMart Digital

Q52 How can ETags help with concurrent updates?

ANSWER

Beyond caching, ETags enable optimistic concurrency for writes. The client sends the ETag it last saw in `If-Match` on an update; if the resource changed since (ETag no longer matches), the server returns **412 Precondition Failed** instead of overwriting — preventing lost updates from concurrent edits.

Q53 How does compression improve API performance?

ANSWER

Enabling gzip or Brotli compression (negotiated via `Accept-Encoding`) shrinks response payloads, reducing transfer time and bandwidth — especially effective for large JSON responses, which compress well. The trade-off is a little CPU, almost always worthwhile for text-based APIs.

Q54 What is the N+1 problem in an API context, and how do you avoid it?

ANSWER

It occurs when serving a list triggers one query per item to load related data — N+1 database round-trips — which slows the endpoint badly at scale. Avoid it by fetching related data in a single query (join/batch), shaping DTOs to what the endpoint needs, and letting clients request expansions explicitly rather than always loading everything.

Q55 What role does a CDN play for APIs?

ANSWER

A Content Delivery Network caches responses at edge locations near users, serving cacheable `GET` responses without hitting the origin — cutting latency and offloading traffic. It works for public, cacheable data (with appropriate `Cache-Control`), while dynamic, user-specific responses bypass it.

Q56 How does pagination improve performance and reliability?

ANSWER

Returning an entire large collection in one response is slow, memory-heavy, and fragile. Pagination returns bounded pages, capping response size and server work, keeping latency predictable, and protecting both server and client from unbounded payloads. Always enforce a maximum page size.

SECTION 08

Documentation & Standards

Q57 What is OpenAPI (Swagger)?

ANSWER

OpenAPI is a standard, language-agnostic specification for describing REST APIs — endpoints, parameters, request/response schemas, and auth. From this machine-readable contract you can generate interactive documentation (Swagger UI), client SDKs, and server stubs, and validate requests. It is the de-facto standard for documenting and sharing REST APIs.

Q58 What is API-first design?

ANSWER

API-first means designing and agreeing the API contract (e.g. an OpenAPI spec) **before** implementing it. Frontend, backend, and consumer teams can then work in parallel against the contract, and mock servers let clients start early. It reduces integration surprises and treats the API as a deliberate product rather than an afterthought.

Q59 What is HAL, and how does it relate to HATEOAS?

ANSWER

HAL (Hypertext Application Language) is a simple, conventional JSON format for embedding hypermedia links (and nested resources) in responses via a `_links` object. It is a concrete way to implement HATEOAS, giving clients a standard place to discover related actions and navigate the API.

Q60 What are the standard content types for REST APIs?

ANSWER

`application/json` is the dominant format. Others include `application/xml`, `application/problem+json` (RFC 7807 errors), `application/hal+json` (hypermedia), and `multipart/form-data` (file uploads). The `Content-Type` header declares the body's format and `Accept` requests the desired response format.

© 2026 ByteMart Digital

Q61 What is RFC 7807 Problem Details?

ANSWER

RFC 7807 defines a standard JSON (or XML) format for HTTP error responses, with fields like `type`, `title`, `status`, `detail`, and `instance`, served as `application/problem+json`. Adopting it gives every error a consistent, machine-readable shape that clients can handle uniformly.

Q62 Why document error responses, not just success responses?

ANSWER

Clients must handle failures as much as successes, so the documentation should specify which status codes and error bodies each endpoint can return and what they mean. Undocumented errors force integrators to discover behaviour by trial and error, leading to brittle client code. A complete contract covers the unhappy paths.

SECTION 09

REST vs Alternatives

Q63 What is the difference between REST and GraphQL?

ANSWER

REST exposes multiple endpoints each returning a fixed resource shape; GraphQL exposes a single endpoint where the client specifies exactly the fields it wants in a query. GraphQL avoids over- and under-fetching and is great for complex, client-driven data needs, but adds server complexity and makes HTTP caching harder — REST's fixed URLs cache naturally.

	REST	GraphQL
Endpoints	many	one
Data shape	fixed per endpoint	client-specified
Caching	native (HTTP)	harder (custom)
Over-fetching	common	avoided

Q64 What is the difference between REST and gRPC?

ANSWER

REST uses HTTP/1.1 and human-readable JSON, ideal for public, browser-facing APIs. gRPC uses HTTP/2 and compact binary Protocol Buffers with a strict contract and streaming support, giving lower latency and payload size — favoured for high-performance internal service-to-service communication. REST wins on ubiquity and debuggability; gRPC on efficiency.

Q65 What is the difference between REST and SOAP?

ANSWER

SOAP is a rigid, XML-based protocol with formal contracts (WSDL) and built-in standards for security and transactions — verbose but strict, still seen in enterprise and financial systems. REST is a lightweight architectural style, typically JSON over HTTP, that is simpler and faster to work with. REST has become the default for web and mobile APIs.

© 2026 ByteMart Digital

Q66 When would you choose REST over the alternatives?

ANSWER

Choose REST for public APIs, browser and mobile clients, and straightforward resource-oriented CRUD, where its simplicity, ubiquity, cacheability, and tooling shine. Consider GraphQL for rich, client-driven data with many relationships, and gRPC for high-throughput internal microservice calls. The right choice depends on the consumers and the data-access patterns.

Q67 What are webhooks, and how do they complement REST?

ANSWER

A webhook is a server-to-client HTTP callback: instead of the client polling a REST endpoint for changes, the server pushes an event to a URL the client registered. Webhooks complement REST by handling asynchronous notifications (payment completed, job finished) efficiently, avoiding wasteful polling — the receiver just needs to verify authenticity and tolerate retries.

Q68 What is the difference between REST and WebSockets?

ANSWER

REST is request/response over short-lived HTTP connections — the client always initiates. WebSockets provide a persistent, full-duplex connection allowing the server to push data to the client in real time. Use REST for standard CRUD and queries; use WebSockets for live, bidirectional needs like chat, live feeds, or collaborative editing.

© 2026 ByteMart Digital

SECTION 10

Advanced Topics

Q69 How do you handle long-running operations in a REST API?

ANSWER

For work that cannot complete within a request, return **202 Accepted** immediately with a URL to a status resource the client can poll (or notify via webhook). The operation runs asynchronously, and the status endpoint reports progress and, on completion, links to the result — keeping the API responsive.

```text
POST /reports        -> 202 Accepted, Location: /reports/jobs/9
GET  /reports/jobs/9 -> 200 { "status": "RUNNING" } ... then "DONE"
```

Q70 What are rate-limit headers, and why include them?

ANSWER

Rate-limit headers tell clients their quota and current usage — commonly `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `X-RateLimit-Reset` (or the standardised `RateLimit-*`). They let well-behaved clients throttle themselves before hitting 429, and the accompanying `Retry-After` tells them when to resume.

Q71 How do you design bulk / batch operations?

ANSWER

For creating or updating many items at once, expose a batch endpoint that accepts an array and returns a per-item result (each with its own status), so a partial failure does not fail the whole batch. Report an aggregate status (e.g. 207 Multi-Status or 200 with a results array) that tells the client exactly which items succeeded.

Q72 What is JSON Patch, and when is it used?

ANSWER

JSON Patch (RFC 6902) is a standard format describing a sequence of operations (add, remove, replace) to apply to a JSON document, sent with `PATCH` and content type `application/json-patch+json`. It expresses precise partial updates unambiguously, which is useful for complex documents where a simple field merge is not enough.

© 2026 ByteMart Digital

```json
[
  { "op": "replace", "path": "/status", "value": "SHIPPED" },
  { "op": "add", "path": "/tags/-", "value": "priority" }
]
```

Q73 What are partial responses / field selection?

ANSWER

Field selection lets a client request only the fields it needs via a query parameter (`?fields=id,name,status`), reducing payload size and giving REST some of GraphQL's flexibility. It is especially valuable for mobile clients on constrained networks, at the cost of extra server logic to shape the response.

Q74 How should an API handle backward compatibility?

ANSWER

Evolve additively: add new optional fields and endpoints without changing or removing existing ones, and never repurpose a field's meaning. Clients must ignore unknown fields. This lets the API grow without breaking existing consumers, reserving a version bump for genuinely breaking changes.

> **WATCH OUT**
> Removing a field, renaming it, tightening validation, or changing a status code are all breaking changes — they require a new version and a deprecation window.

Q75 What is the difference between soft delete and hard delete in an API?

ANSWER

A **hard delete** permanently removes the resource (a `DELETE` returning 204). A **soft delete** marks it inactive/archived while retaining the data, so it can be restored or audited; the resource then disappears from normal listings. Soft deletes suit data with legal, audit, or recovery requirements.

Q76 How do you secure file uploads in a REST API?

ANSWER

Validate the file type and size, scan for malware, store outside the web root (or in object storage) with generated names, and never trust the client-supplied filename or content type. Return a resource reference rather than serving the raw file back directly. Uploads are a common attack vector, so treat every uploaded file as hostile.

SECTION 11

HTTP Semantics Deep-Dive

Q77 What is the difference between 301, 302, 307, and 308 redirects?

ANSWER

All are redirects, differing in permanence and method handling. **301** permanent and **302** temporary historically allowed clients to change the method to GET; **308** (permanent) and **307** (temporary) preserve the original method and body. Use 301/308 when a resource has permanently moved, 302/307 for temporary redirection.

Code	Permanent	Keeps method
301	yes	not guaranteed
302	no	not guaranteed
307	no	yes
308	yes	yes

Q78 What is the purpose of the Location header?

ANSWER

The `Location` header tells the client where a resource is. After a **201 Created** it points to the URL of the newly created resource so the client can fetch it; in a redirect (3xx) it gives the URL to follow. Returning `Location` on creation is a REST best practice.

```text
POST /orders  -> 201 Created
Location: /orders/42
```

Q79 What are Range requests and 206 Partial Content?

ANSWER

A client can request part of a resource with the `Range` header (e.g. a byte range of a large file), and the server responds **206 Partial Content** with just that portion. This enables resumable downloads and media streaming without transferring the whole resource.

Q80 Is DELETE idempotent even though the resource disappears?

ANSWER

Yes. Idempotency is about the **effect on server state**, not the response code. The first `DELETE` removes the resource (204); a repeat has no further effect (often 404 or 204). Since the state — resource gone — is the same after one or many calls, `DELETE` is idempotent, which is what lets clients safely retry it.

Q81 What is the difference between query parameters and path parameters?

ANSWER

Path parameters identify a specific resource and are part of its address (`/orders/42`). **Query parameters** refine or modify a request without identifying a different resource — filtering, sorting, pagination (`/orders?status=paid`). Put identity in the path and options in the query string.

Q82 What are the key request and response headers in a REST call?

ANSWER

- **Authorization** — carries credentials or a bearer token.
- **Content-Type** — the format of the request/response body.
- **Accept** — the response format the client wants.
- **Cache-Control / ETag** — caching and revalidation.
- **Location** — URL of a created or redirected resource.

© 2026 ByteMart Digital

SECTION 12

Design & Best Practices Extras

Q83 What is over-fetching and under-fetching?

ANSWER

Over-fetching is receiving more data than needed (a fixed endpoint returns fields the client discards); **under-fetching** is receiving too little, forcing extra calls to assemble what is needed. REST mitigates both with field selection, resource expansion (`?expand=customer`), and well-designed endpoints; GraphQL avoids them by design.

Q84 How do you expose related resources — embed or link?

ANSWER

You can **link** to related resources (return an id or URL the client fetches separately) or **embed** them inline. Linking keeps responses small and cacheable but adds round-trips; embedding (often opt-in via `?expand=`) saves calls at the cost of larger payloads. Offering embedding as optional gives clients control.

Q85 What is a deprecation strategy for an API?

ANSWER

Deprecate gracefully: announce ahead of time, signal it in responses (a `Deprecation` or `Sunset` header and documentation), keep the old version running through a migration window, and monitor remaining usage before removal. Abrupt removal breaks integrations and erodes trust.

Q86 How should sorting by multiple fields be expressed?

ANSWER

Use a repeatable or comma-separated `sort` parameter with an explicit direction per field, e.g. `?sort=status,asc&sort;=createdAt,desc`. A clear, consistent convention lets clients express compound ordering unambiguously, and the server should validate sortable fields to avoid exposing arbitrary columns.

© 2026 ByteMart Digital

Q87 What is the difference between a public and an internal API's design priorities?

ANSWER

A **public** API prioritises stability, backward compatibility, thorough documentation, security, and rate limiting, because external consumers depend on it and cannot be forced to change. An **internal** API can evolve faster and optimise for performance and team velocity, since consumers are known and can coordinate upgrades.

Q88 How do you return metadata alongside a collection?

ANSWER

Attach pagination and summary metadata either in the response body (an envelope with a `meta` object) or in headers (`Link` for navigation, `X-Total-Count` for totals). Either is acceptable; the important thing is applying one approach consistently so clients always know where to find navigation information.

Q89 What is mass assignment, and why is it dangerous?

ANSWER

Mass assignment happens when an API binds the entire request body onto an entity, letting a client set fields it should not — like `role` or `isAdmin` — by including them in the payload. Prevent it by binding to a purpose-built DTO that exposes only editable fields, never directly to the persistence entity.

> **WATCH OUT**
> Binding request bodies straight onto entities is a classic privilege-escalation vulnerability — always map through an explicit input DTO.

© 2026 ByteMart Digital

SECTION 13

Security, Testing & Operations

Q90 What are the top REST API security risks (OWASP)?

ANSWER

- **Broken object-level authorization** — accessing others' data by id.
- **Broken authentication** — weak or missing token validation.
- **Excessive data exposure** — returning more fields than needed.
- **Lack of rate limiting** — enabling abuse and DoS.
- **Injection** and **mass assignment** — untrusted input reaching logic or state.

Q91 What are important HTTP security headers?

ANSWER

Beyond auth, responses should set protective headers — `Strict-Transport-Security` (force HTTPS), `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, and appropriate CORS headers. They harden the API and its browser clients against downgrade, sniffing, and cross-site attacks.

Q92 How do refresh tokens work with access tokens?

ANSWER

Access tokens are short-lived to limit exposure; when one expires, the client uses a longer-lived **refresh token** to obtain a new access token without re-authenticating the user. The refresh token is stored securely and can be revoked, giving both good UX and the ability to cut off access — mitigating the JWT non-revocation problem.

Q93 What are OAuth2 scopes?

ANSWER

Scopes are named permissions attached to an access token (e.g. `orders:read`, `orders:write`) that limit what the token can do. They enforce least privilege — a client is granted only the scopes it needs — and the API checks the required scope on each endpoint before allowing the action.

© 2026 ByteMart Digital

Q94 How do you test a REST API?

ANSWER

Test at several levels: unit tests for logic, integration tests hitting the endpoints (with MockMvc, RestAssured, or a test client) asserting status codes, headers, and body, and contract tests verifying the API matches consumer expectations. Cover the unhappy paths — validation errors, auth failures, not-found — not just the happy case.

```java
mockMvc.perform(get("/orders/1").header("Authorization", bearer))
    .andExpect(status().isOk())
    .andExpect(header().exists("ETag"))
    .andExpect(jsonPath("$.status").value("PAID"));
```

Q95 What is the Retry-After header, and when is it used?

ANSWER

`Retry-After` tells a client how long to wait before retrying, given as seconds or a date. Servers send it with **429** (rate limited) and **503** (temporarily unavailable) so clients back off intelligently instead of hammering a busy or throttled service.

Q96 How do you monitor and observe a REST API in production?

ANSWER

Track the RED signals — request rate, error rate, and latency (percentiles) — per endpoint, emit structured logs with a correlation id, and expose health and metrics endpoints, and alert on error-rate and latency thresholds. This observability is what lets you detect regressions and diagnose issues quickly.

Q97 What is a contract test, and why is it valuable for APIs?

ANSWER

A contract test verifies that the API's actual behaviour matches an agreed contract that consumers rely on — fields, types, status codes. Run in the provider's pipeline, it catches breaking changes before release without the fragility of full end-to-end tests, which is essential when producer and consumers deploy independently.

© 2026 ByteMart Digital

SECTION 14

Practical Scenarios

Q98 Why is returning 200 with an error in the body an anti-pattern?

ANSWER

Signalling a failure with **200 OK** and an error flag in the body breaks HTTP semantics: caches, retries, and monitoring all treat the response as success, so failures go undetected and clients must parse the body to know something went wrong. Use the correct 4xx/5xx status so the whole ecosystem reacts appropriately.

> **WATCH OUT**
> The status code is the contract for success or failure — never bury an error inside a 200 response.

Q99 What is the difference between an endpoint and a resource?

ANSWER

A **resource** is the conceptual thing your API exposes (an order, a user). An **endpoint** is a specific URL plus method combination that acts on a resource (`GET /orders/42`). One resource is typically served by several endpoints — the collection, the item, and its methods.

Q100 How do you include navigation links for pagination?

ANSWER

Provide links to the next, previous, first, and last pages so clients navigate without constructing URLs themselves — either in the response body (a `_links` object) or in the standard `Link` header. This is HATEOAS applied to collections and makes paging robust against URL changes.

```text
Link: </orders?page=2>; rel="next", </orders?page=0>; rel="first"
```

Q101 What is the difference between REST and RPC styles?

ANSWER

RPC models the API as actions/verbs (`/createOrder`, `/cancelOrder`) invoked like remote functions. **REST** models it as resources acted on with standard HTTP methods (`POST /orders`, `POST /orders/42/cancel`). REST leverages HTTP semantics and caching; RPC can be more natural for action-oriented operations that do not map to resources.

Q102 When should an operation return 202 Accepted versus 200/201?

ANSWER

Return **200/201** when the work is finished by the time you respond. Return **202 Accepted** when the request has been accepted for asynchronous processing that is not yet complete, providing a way to track its progress. 202 sets the client's expectation that the outcome is pending.

Q103 What is a correlation / request id, and how is it used in APIs?

ANSWER

A request id (e.g. an `X-Request-Id` header) uniquely identifies a request across logs and services. Accepting one from the client or generating it at the edge, echoing it in the response, and logging it everywhere lets you trace a single request end to end when diagnosing problems — invaluable for support and debugging.

Q104 How do HTTP/2 and HTTP/3 benefit REST APIs?

ANSWER

HTTP/2 adds multiplexing (many requests over one connection), header compression, and server push, reducing latency versus HTTP/1.1's connection limits. HTTP/3 runs over QUIC (UDP), improving performance on lossy networks and speeding connection setup. Both benefit chatty API clients without changing the REST design itself.

Q105 What is connection reuse (keep-alive), and why does it matter?

ANSWER

Persistent connections (keep-alive) let multiple requests reuse one TCP/TLS connection instead of paying the handshake cost each time. For clients making many API calls, connection reuse via pooling substantially cuts latency and resource usage — which is why HTTP clients should share a pooled connection manager rather than opening a new connection per call.

Q106 What is the JSON:API specification?

ANSWER

JSON:API is an opinionated standard for building JSON APIs, defining conventions for document structure, relationships, includes, pagination, filtering, and errors. Adopting it gives consistency and interoperable tooling out of the box, at the cost of some rigidity — useful when you want a well-defined convention rather than designing every detail yourself.

© 2026 ByteMart Digital

Q107 How should an API handle a partial failure in a batch request?

ANSWER

Rather than failing the entire batch on one bad item, process each independently and return a per-item result with individual statuses — often an overall **207 Multi-Status** or a 200 with a results array. The client can then see exactly which items succeeded and retry only the failures.

Q108 What is the Idempotency of PUT versus PATCH in practice?

ANSWER

PUT is idempotent because sending the same full representation repeatedly leaves the resource in the same final state. PATCH is idempotent only if the patch itself is — replacing a field is idempotent, but an operation like 'increment' or 'append' is not. Design PATCH payloads to be idempotent where possible so retries are safe.

Q109 How do you design an API to be evolvable over time?

ANSWER

Favour additive change, use DTOs to decouple the wire format from internal models, have clients ignore unknown fields, document a clear versioning and deprecation policy, and lean on hypermedia so URLs can change without breaking clients. Designing for change from the start avoids painful, breaking rewrites later.

> **KEY IDEA**
> Evolvability comes from loose coupling: additive changes, tolerant clients, DTO boundaries, and a clear versioning policy.

Q110 How do you localize a REST API's responses?

ANSWER

Use the `Accept-Language` request header to signal the client's preferred language, and return localized messages (and a `Content-Language` header) accordingly. Keep data neutral (ISO dates, codes) and translate only human-facing text like error messages and labels, falling back to a default language when a translation is missing.

© 2026 ByteMart Digital

Q111 What is the difference between 200 and 204 for an update?

ANSWER

Return **200 OK** with the updated resource in the body when the client benefits from seeing the new state (e.g. server-computed fields). Return **204 No Content** when there is nothing useful to send back and the client already knows the result. Both are valid; pick based on whether the response body carries value.

Q112 What is the difference between a synchronous and an asynchronous API?

ANSWER

A **synchronous** API completes the work and returns the result within the request. An **asynchronous** API acknowledges the request (202 Accepted) and does the work in the background, letting the client poll a status resource or receive a webhook. Async suits slow or long-running operations that would otherwise time out the request.

Q113 What is a well-known URI?

ANSWER

Well-known URIs (under `/.well-known/`, per RFC 8615) are standardized paths where clients expect to find specific metadata — for example `/.well-known/openid-configuration` for OIDC discovery. They let clients locate configuration or capabilities in a predictable, conventional place without prior arrangement.

SECTION 15

Coding Challenges

Q114 Design a paginated GET endpoint that returns items with metadata. </> CODE

SOLUTION

Accept page and size query parameters (capping size), return the page of items plus pagination metadata so the client can navigate.

java

```java
@GetMapping("/orders")
public PagedResponse<OrderDto> list(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {
    int capped = Math.min(size, 100);
    Page<Order> result = repo.findAll(PageRequest.of(page, capped));
    return new PagedResponse<>(
        result.map(this::toDto).getContent(),
        result.getNumber(), result.getTotalPages(),
        result.getTotalElements());
}

public record PagedResponse<T>(List<T> items, int page,
                               int totalPages, long totalItems) { }
```

Q115 Implement an idempotent POST using an idempotency key. </> CODE

SOLUTION

Read the client's key, return the stored result if the key was seen before, otherwise perform the action and record the result against the key.

java

```java
@PostMapping("/payments")
public ResponseEntity<PaymentDto> pay(
        @RequestHeader("Idempotency-Key") String key,
        @Valid @RequestBody PaymentRequest req) {
    return idempotencyStore.find(key)
        .map(saved -> ResponseEntity.ok(saved))
        .orElseGet(() -> {
            PaymentDto result = paymentService.charge(req);
            idempotencyStore.save(key, result);
            return ResponseEntity.status(201).body(result);
        });
}
```

© 2026 ByteMart Digital

Q116 Return a standardized RFC 7807 error response.

</> CODE

SOLUTION

Use Spring's `ProblemDetail` in a global handler to produce a consistent `application/problem+json` body.

java

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(NotFoundException.class)
    public ProblemDetail handleNotFound(NotFoundException e) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, e.getMessage());
        pd.setType(URI.create("https://api.bytemart.com/errors/not-found"));
        pd.setTitle("Resource not found");
        return pd;   // served as application/problem+json
    }
}
```

Q117 Implement a conditional GET with an ETag returning 304.

</> CODE

SOLUTION

Compute the resource's ETag, compare it to the client's `If-None-Match`, and return 304 with no body when unchanged.

java

```
@GetMapping("/orders/{id}")
public ResponseEntity<OrderDto> get(
        @PathVariable Long id,
        @RequestHeader(value = "If-None-Match", required = false) String inm) {
    Order order = service.get(id);
    String etag = "\"" + order.version() + "\"";
    if (etag.equals(inm))
        return ResponseEntity.status(HttpStatus.NOT_MODIFIED).eTag(etag).build();
    return ResponseEntity.ok().eTag(etag).body(toDto(order));
}
```

Q118 Validate a request body and return field-level errors.

</> CODE

SOLUTION

Constrain the request record and handle the validation exception to produce a structured 422 with per-field messages.

```java
public record CreateUser(@NotBlank String name,
                         @Email String email,
                         @Min(18) int age) { }

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ProblemDetail> onInvalid(MethodArgumentNotValidException e) {
    ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.UNPROCESSABLE_ENTITY);
    pd.setTitle("Validation failed");
    pd.setProperty("errors", e.getBindingResult().getFieldErrors().stream()
        .map(f -> Map.of("field", f.getField(),
                         "message", f.getDefaultMessage())).toList());
    return ResponseEntity.unprocessableEntity().body(pd);
}
```

Q119 Expose a versioned endpoint via the URI.

</> CODE

SOLUTION

Namespace by version in the path so a breaking v2 can coexist with v1 during migration.

```java
@RestController
@RequestMapping("/v2/orders")
public class OrderV2Controller {
    @GetMapping("/{id}")
    public OrderV2Dto get(@PathVariable Long id) {
        return service.getV2(id);   // new shape; /v1/orders stays available
    }
}
```